

CSCI 232

232 Algorithms & Data Structures

LECTURE: 6

J. L. Pach

OUTLINE:

- Structs
- typedef

STRUCTURES

STRUCTURES

```
Syntax:
struct symbolic_name1
{
        <statement1>
}<symbolic_name2, ...>;
```

- Structures are user-defined data types that group together variables of different data types.
- A structure is a collection of one or more variables, possibly of different types, grouped together under single `symbolic_name` for convenient handling.
- Everything that is in angle brackets <> is optional.

TYPEDEF & STRUCT 1/1

- Normally, when we define a struct, we can also declare global variables or pointers immediately after the closing brace:

```c
struct Point
{
    int x, y;
} p1, *ptr;
```

- The variables `p1` and `ptr` are global objects of this structure type.

- We can also **omit the structure tag name** to make it **anonymous**, preventing multiple instances from being created:

```c
struct
{
    int id;
    float value;
} config;
```

- Here, config is the **only instance** of this unnamed structure type.

TYPEDEF & STRUCT 2/2

- Alternatively, we can use `typedef` to create an **alias** for a structure type:

```c
typedef struct
{
    int x, y;
} Point;
int main()
{       //...
        Point symbolic_name;
}
```



- Now we can declare variables as Point p1; — no need for the keyword struct.
- The name after the definition (`Point`) **is a type alias, not a variable.**
- **This alias does not prevent creating multiple instances** — it simply simplifies the syntax.

TYPEDEF & STRUCT - SUMMARY

Form	Description	Example
`struct Tag { ... } var;`	Defines type + creates variable	`struct Point {int x; int y;} p;`
`struct { ... } var;`	Anonymous struct, single instance	`struct {int x;} s;`
`typedef struct { ... } Alias;`	Creates a reusable type alias	`typedef struct {int x;} Point;`

AN EXAMPLE

```c
#include <stdio.h>
struct MyStruct
{
    int value;
}*p, s; /* like struct MyStruct  * sPointer; struct MyStruct myStructure;*/

struct MyStruct  * sPointer; /* global pointer */
struct MyStruct myStructure; /* global variable */

struct MyStruct function( struct MyStruct temp )
{
    temp.value += 5;
    return temp;
}
```

```c
int main(int argc, char *argv[])
{
    struct MyStruct localStruct; /* local variable */
    localStruct.value = 1;
    struct MyStruct * localStructPointer; /* local pointer */
    localStructPointer = &localStruct;
    printf( "%d\n", localStruct.value );                                    // 1
    localStructPointer->value = 2;
    printf( "%d\n", localStructPointer->value );                            // 2
    printf( "%d\n", (*localStructPointer).value ); /*Equivalent to the previous line*/  // 2
    p = & s;
    (*p).value = 2;
    printf( "%d\n", s.value );                                          // 2
    printf( "%d\n", p->value );                                             // 2

    printf( "%d\n", myStructure.value );                                    // 0
    myStructure = function( *localStructPointer );
    printf( "%d\n", myStructure.value );                                    // 7
    return 0;
}
```

SUMMARY 1/3

- To access members of a structure, we use the dot (`.`) operator.
- When accessing a member through a pointer, we must use the dereferencing operator (`*`) followed by the member access operator (`.`), enclosed in parentheses: `(*symbolic_name).field`.
- Alternatively, we can use the arrow operator `->`, which is equivalent to `symbolic_name->field`.

SUMMARY 2/3

- Structures are user-defined data types that group together variables of different data types. They provide a way to create custom data types tailored to specific needs.
- **Components:** Structures consist of members (or fields), which can be of different data types.
- **Member access:**
  - **Direct:** To access members of a structure directly, use the dot operator (`.`).
  - **Indirect (through a pointer):** To access members through a pointer, use either:
    - the dereferencing operator (`*`) followed by the member access operator (`.`), enclosed in parentheses: `(*pointer).member`,
    - the arrow operator (`->`): `pointer->member`.

SUMMARY 2/3

- **Pointers to structures:** You can create pointers to structures, similar to other data types. This allows dynamic memory allocation for structures and passing them as function arguments.
- **Global vs. local variables:** Structures can be declared as global variables (accessible from anywhere in the program) or local variables (accessible only within a specific block of code). You can also create pointers to structures for more flexible memory management.

THANK

You